Smart India Hackathon 2026 · Problem Statement 26155 · NTRO

NIRIKSHAK

Technical Report

A vendor-agnostic network security compliance auditor. Every figure in this report is derived from the code at commit `28ab52b`, and every claim names the file that proves it.

Team Atlantis · RMK Engineering College

> How to use this report: find the question in the contents, go to the section, open the file in `monospace`. The index at the end maps each concept to its file.

Contents

1. Tech stack, with reasoning 3
2. Algorithms and AI components 5
    Where AI is used, and where it is structurally forbidden 6
3. Code walkthrough 8
4. Data model 10
5. Testing and verification 12
6. Feasibility, viability and impact 14
7. Known limitations 15
8. Index: concept to file 17

1. Tech stack, with reasoning

Versions are those pinned in `requirements.lock` for the core environment, and the ranges in `pyproject.toml` and `ui/package.json` for the optional extras and the interface. Where an Architecture Decision Record (ADR, in `docs/adr/`) records the alternative that was considered, it is cited.

TECHNOLOGY	USED FOR	CHOSEN OVER, AND WHY
NIRIKSHAK's own block parser `api/parse/block_parser.py`	Configuration text to a `ConfigTree`	**TextFSM with ntc-templates** (ADR 0004). TextFSM targets flat, record-oriented `show`-command output and has no block model. `exec-timeout 10 0` is a management idle timeout under `line vty 0 4` and a different control under `line con 0`; a parser that cannot see the enclosing block cannot tell them apart. Running configurations are hierarchical, so the project builds a tree first and lets patterns run against it. Neither TextFSM nor ntc-templates was ever imported (ADR 0040).
No device-access library	—	**Netmiko and NAPALM**, removed (ADR 0001). Both open live SSH sessions to devices. NIRIKSHAK audits exported configuration files and never connects to a device, so it needs no production credentials and can run on an air-gapped network. `tests/architecture/test_no_device_libraries.py` fails if any device-connection library is imported.
WeasyPrint 63 + Jinja2 `api/report/`	HTML and PDF reports from one template	**ReportLab / FPDF** (ADR 0006, ADR 0015). One Jinja2 template (`api/report/templates/report.html.j2`) renders the HTML report, and WeasyPrint renders the same document to PDF, so the two formats cannot drift. Substituting an engine was recorded as "not recommended without cause"; the cost is the GTK3 system runtime, documented in the README. `tests/architecture/test_report_boundaries.py` fails if `reportlab`, `fpdf`, `pdfkit`, `wkhtmltopdf`, `xhtml2pdf` or `playwright` appears in `api/report/`.
SQLite, standard-library `sqlite3` `api/db/`	Operational store and audit store	**An ORM, a migration framework, a distributed ledger** (ADR 0007: "Standard library `sqlite3` only"). Forward-only, checksum-verified SQL migrations (`api/db/migrate.py`). Two separate database files (ADR 0009, decision D4), so that "the audit database contains no configuration content" is provable by opening the file, and the operational store could be encrypted without touching the chain's verifiability. WAL mode (`api/db/connection.py`).
sentence-transformers, `all-MiniLM-L6-v2` `api/learn/embedding.py`	Embedding unknown lines for similarity ranking	A small CPU model (384 dimensions, `EMBEDDING_DIMENSIONS`) that runs offline once downloaded (ADR 0017, ADR 0018). **No fallback**: if the model is unavailable the adapter raises rather than substituting a hash or bag-of-words embedding that would look like model output. Optional extra `[ai]`.

FAISS (`faiss-cpu`)	**Installed by `[ai]`, not imported**	Retrieval today is exact cosine similarity over every labelled example (`cosine`, `rank_candidates` in `api/learn/suggest.py`), written out so it is testable without the extra. With the index at its current size, exact search is correct and cheap. An approximate index such as FAISS is where retrieval would move if the labelled index grew large (section 6).
YAML, validated by Pydantic and JSON Schema	Vendor packs, rules, framework indexes, snippets	DEVELOPER_GUIDE.md Rule 5: rules and vendor packs are data, so adding a vendor, framework or OS version is a data change. Every file loads through a typed contract (`api/models/`); snippets are also validated against `snippets/schema/snippet.schema.json` (`api/remediate/library.py`).
Python 3.11, FastAPI 0.115, Pydantic 2.13, Uvicorn 0.32	The API and every contract	Python 3.11 in a project-local virtual environment (ADR 0002). FastAPI and Pydantic are fixed by the project constraints (DEVELOPER_GUIDE.md §11); no ADR records an alternative that was weighed, so none is claimed here. Every inter-stage contract is a frozen Pydantic model with `extra="forbid"`.
React 18, TypeScript 5.6, Vite 5, Tailwind 3 `ui/`	The interface	Recorded in ADR 0022 as the interface stack; the ADR records the design principles, not an alternative framework weighed against React, so none is claimed. The interface is a pure consumer of the API. Fonts are vendored so an air-gapped deployment makes no external request.
lxml 5.4	XML well-formedness during format detection; reading the DISA STIG XCCDF	`api/ingest/format_detect.py`, `scripts/fetch_framework_catalog.py`. There is no XML *configuration* parser (section 7).
`hashlib.scrypt`, `hashlib.sha256`	Password hashing; the audit hash chain	Standard library, no invented cryptography (`api/security/passwords.py`, `api/models/audit.py`).

2. Algorithms and AI components

Each entry gives what it does, where it lives, why this approach, and what it is deliberately not allowed to do.

2.1 Block parser

Does: turns configuration text into a `ConfigTree` of `ConfigNode`s with parent/child context, plus `UnplacedLine`s for everything that is not a command. **Where:** `api/parse/block_parser.py`. **Syntax modes:** the contract names five (`SyntaxMode`, `api/models/enums.py`): indentation, brace nesting, flat `set` paths, XML and JSON. Three are implemented (`IMPLEMENTED_MODES`); XML and JSON raise `UnsupportedSyntaxModeError` rather than returning an empty tree. **Four invariants**, checked by the conformance suite: **lossless** (reconstruction reproduces the source exactly), **evidence** (every node yields a complete `Evidence`), **total** (every line is a node or an `UnplacedLine`, never dropped), **deterministic** (same bytes in, same tree out). **Not allowed:** comments, blank lines and literal block bodies (banners, keys, certificates) never become nodes, so a commented-out directive or a banner reading "ip ssh version 1 is prohibited" can never produce a fact.

2.2 Vendor detection

Does: scores each pack by summing the weights of its `detect` signatures that match the file (`score_pack`), then applies two thresholds (`detect_vendor`, `api/ingest/vendor_detect.py`): the winner must reach `detection_min_score` 0.60 and lead the runner-up by `detection_min_margin` 0.25 (`api/config.py`). **Why two:** thin evidence and ambiguous evidence are different failures with different fixes; both produce UNKNOWN, with different recorded outcomes (`BELOW_THRESHOLD`, `AMBIGUOUS`). **Not allowed:** no model, no if-chain of vendor names; a guessed platform would apply the wrong parsing rules to every line.

2.3 Pattern engine and per-field merge policy

Does: applies a pack's anchored regexes, optionally scoped to a block header, to tree nodes and produces `FieldMatch`es, and collects unmatched nodes as **residue** (`api/parse/pack_engine.py`, `api/parse/service.py`). `build_field` (`api/parse/fields.py`) turns matches into one canonical field. When two lines disagree, `_resolve_disagreement` consults `FIELD_MERGE_POLICY` (`api/models/csm.py`): only `telnet_enabled` opts in, as `WORST_CASE_TRUE`, because one vty range permitting telnet makes the device reachable by telnet however many refuse it. **Not allowed:** every other field abstains on disagreement (UNKNOWN, `conflicting_evidence`); two different integers are never made comparable by declaring one worse.

2.4 Canonical model and absence resolution

Does: `build_csm` (`api/normalise/service.py`) produces a `CanonicalSecurityModel`, deciding what every *absent* directive means (`api/normalise/absence.py`): the platform's documented default (`ABSENT_DEFAULT`), a control the platform cannot express (`ABSENT_UNSUPPORTED`), or an honest `UNKNOWN`. **Why:** a missing line may be the platform default or a removed control, opposite conclusions from identical evidence. **Not allowed:** a default is only usable if it cites vendor documentation, and none ships, so today every absent field resolves to UNKNOWN (`capability_unknown`). `on_capability_unknown` can only be UNKNOWN; the contract refuses any other value (`AbsencePolicy`, `api/models/rule.py`).

2.5 Deterministic rule engine

Does: `evaluate_device` (`api/comply/engine.py`) produces one `Finding` per applicable rule, in rulepack order. Rules are YAML (`rules/canonical/`): one field, and either one condition or an `all_of` conjunction (`CheckSpec`, `api/models/rule.py`). **Closed operator set** of twelve (`ConditionOp`): equals, not_equals, gt, gte, lt, lte, in, not_in, contains, is_true, is_false, non_empty. **Why:** an expression language is where vendor logic and model calls would reappear. **Not allowed:** no disjunction, negation or nesting ("a rule needing disjunction is two rules"); a condition that cannot be evaluated against its value abstains with `rule_type_mismatch` and never becomes FAIL; a verdict without evidence abstains with `no_evidence`. The rulepack loads only if its checksum matches `rules/rulepack.yaml` (`load_rulepack`, `api/comply/rulepacks.py`).

2.6 ACL interval analysis

Does: reports **shadowed**, **redundant** and **overly permissive** entries by interval containment over addresses, ports and protocols (`api/analyse/intervals.py`, `api/analyse/acl_analysis.py`). Order is the semantics: first match wins. **Why:** interval logic is checkable; coverage is pairwise ("line 40 can never fire because of line 20"), which under-reports rather than over-reports. **Not allowed:** containment is three-valued; an unresolved object-group is *unknown*, not empty, and becomes an UNDETERMINED observation rather than silently dropping out. Not an LLM. Observations are a separate rail from verdicts.

2.7 Token-shape clustering

Does: replaces the varying parts of a residue line with typed placeholders (`ntp server 192.0.2.20` → `ntp server <IP>`, `api/learn/signature.py`) and groups lines with identical signatures, ranked by frequency then first appearance (`api/learn/cluster.py`). **Why:** the cluster is the unit an administrator confirms, so one decision applies to every line in it. **Not allowed:** it is exact string grouping, not a model; a fuzzy cluster would let one confirmation cover lines the administrator never saw.

2.8 Embedding and retrieval

Does: embeds each cluster representative and the labelled index with `all-MiniLM-L6-v2`, normalised at source (`embed`, `api/learn/embedding.py`), and ranks candidates by cosine similarity, ties broken by field then text, returning at most `MAX_SUGGESTIONS` = 3 (`rank_candidates`, `api/learn/suggest.py`). The index holds only two kinds of labelled example: pack examples from the development split (`SEED`) and administrator decisions (`ADMIN`) (`api/learn/index.py`). **Not allowed:** no fallback embedding; no invented examples; under airgap the loader fails closed rather than fetching weights.

2.9 The calibration refusal

Does: holds the machinery to map a raw similarity score to a probability of being correct, and refuses to fit: `MIN_CALIBRATION_SAMPLES` = 200 and `MIN_POSITIVE_SAMPLES` = 20 (`api/learn/calibration.py`). **Why:** the required population, line-level ground truth, does not exist. **Not allowed:** no calibrator ships, so every suggestion is `UNCALIBRATED_SIMILARITY`, which forces the field to UNKNOWN whatever the score. A similarity score is a ranking, never a probability.

2.10 Pattern compiler

Does: turns one confirmed line into one pack pattern: tokenise, replace the token the administrator nominated with `(\S+)`, escape the rest, anchor with `^` (`tokenise`, `build_regex`, `compile_pattern`, `api/train/compile.py`). Block scope defaults to the literal-escaped header; numeric generalisation is an explicit option (`build_scope`). The pack then moves DRAFT → VALIDATED → ACTIVE (`api/train/activation.py`). **Not allowed:** no inferred alternation, no optional tokens, no merging of two confirmations, no guessing which token is the value; nothing enters a pattern that a named person did not confirm. Built-in packs are never edited at runtime; an activation record in `packs/trained/` names the version in force.

2.11 Where AI is used, and where it is structurally forbidden

Used, in exactly one place: `api/learn/` embeds unknown lines and ranks up to three candidate canonical fields for an administrator to review. That is the only model in the system. No generative model, local LLM or chatbot is used anywhere.

Forbidden, and how the prohibition is enforced:

- **No verdict.** `comply` may not import `learn` or `train`, nor `parse`, so no vendor syntax or model output can reach the rule engine. Checked by static import scan over every module (`test_forbidden_import_edge`, `tests/architecture/test_import_rules.py`, 75 forbidden edges in total).

- **No field value.** `normalise` may not import `learn` or `train`, and a `Suggestion` has no value field — only rank, field, raw score and confidence method (`api/models/training.py`). A suggestion cannot enter the canonical model.
- **No abstention override.** `UNCALIBRATED_SIMILARITY` forces UNKNOWN; a calibrated method cannot be claimed without a fitted calibrator (`api/learn/calibration.py`).
- **No persistence.** `learn` may not import `db`; only `train` composes the advisory branch with storage.
- **No remediation.** `remediate` may not import `learn`; commands come only from `snippets/`, each naming the person who vetted it, and a test refuses a vetter whose name looks automated (`tests/architecture/test_rule_content_policy.py`).
- **No model actor in the log except as a suggester.** The audit schema's `CHECK (actor_type <> 'model' OR action = 'ai_suggested')` rejects any other model-attributed record below Python (`api/db/migrations/audit/0001_initial.sql`).

The one path from a suggestion to a verdict passes through a person: the administrator confirms → `compile_pattern` writes a deterministic regex → the pack is activated → the next parse matches it with confidence method `DETERMINISTIC`.

3. Code walkthrough

`api/` holds 15 packages and 112 modules. "Forbidden" lists the packages each may not import, from `FORBIDDEN_EDGES` in `tests/architecture/test_import_rules.py`. `models` is a leaf and may import nothing else from `api/` (`tests/architecture/test_models_are_leaf.py`).

PACKAGE	OWNS	KEY MODULES	FORBIDDEN TO IMPORT
`models`	Typed contracts every stage speaks	csm, field, evidence, finding, rule, pack, config_tree, audit, training, enums	everything (leaf)
`ingest`	Upload, validation, format and vendor detection, blobs, line cache, identity, pack loading and checksums	service, vendor_detect, format_detect, validate, archive, blobs, packs, pack_checksum, device_identity	comply, learn, normalise, prioritise, remediate, train
`parse`	Text to tree; patterns to field matches; residue	block_parser, pack_engine, fields, structures, service	comply, learn, normalise, prioritise, remediate, train
`normalise`	Parse result to canonical model; absence; residue scrubbing	service, absence, identity, residue	comply, learn, prioritise, remediate, report, train
`comply`	Deterministic rule engine; rulepack loading; framework indexes and scope	engine, conditions, rulepacks, frameworks, service	analyse, ingest, learn, normalise, parse, prioritise, remediate, report, train
`analyse`	ACL interval analysis	intervals, acl_analysis, service	comply, ingest, learn, normalise, parse, prioritise, remediate, report
`prioritise`	Exposure and peer baselines	exposure, baseline, service	audit, comply, db, ingest, learn, normalise, parse, remediate, report, train
`remediate`	Vetted snippet library and resolver	library, resolver	analyse, comply, ingest, learn, normalise, parse, prioritise, report
`report`	View model, HTML render, PDF probe and render	model, render, pdf, templates/ report.html.j2	comply, db, ingest, learn, normalise, parse, train
`learn`	Advisory branch: signatures, clusters, embeddings, index, suggestions, calibration	signature, cluster, embedding, index, suggest, calibration	analyse, comply, db, normalise, prioritise, remediate, report, train
`train`	Confirmation loop: queue, compiler, pack lifecycle	service, queue, compile, activation	analyse, comply, remediate, report
`audit`	Append-only hash chain: append, read, verify	chain, store, verify	comply, learn, parse
`db`	Connections, migrations, row stores	connection, migrate, findings, training, users	—
`security`	Password hashing; secret scrubbing before inference	passwords, scrub	—
`routers`	The HTTP surface: I/O and authorisation	ingest, audits, reports, training, fleet, audit, users, deps	—

3.1 One audit request, end to end

`POST /compliance/audits?file_id=…&framework=…` → `run_audit_endpoint` (`api/routers/audits.py`):

1. `resolve_selection` — refuses an unsourced framework with 400 (`api/comply/frameworks.py`).

2. Ownership check (`require_access`, `api/routers/deps.py`); `find_pack` for the detected platform (`api/ingest/packs.py`); 409 if none.
3. `blobs.read` — the stored bytes, by content hash (`api/ingest/blobs.py`).
4. `parse_configuration` — tree, field matches, residue (`api/parse/service.py`).
5. `extract_identity` — hostname, OS version, model where declared (`api/ingest/device_identity.py`).
6. `build_csm` — the canonical model (`api/normalise/service.py`).
7. `scope_selection` — frameworks that describe this device; 409 if none does (`api/comply/frameworks.py`).
8. `record_residue` — the training queue for this file (`api/train/service.py`).
9. `evaluate_device` — findings (`api/comply/engine.py`).
10. `analyse_device` — ACL observations (`api/analyse/service.py`); `prioritise` — exposure, abstaining when undetermined (`api/prioritise/service.py`).
11. `save_run` — run and findings persisted (`api/db/findings.py`).
12. `AuditChain.append_system` with `AUDIT_RUN` and `audit_payload` — counts, identifiers, versions, rulepack checksum (`api/audit/chain.py`, `api/comply/service.py`).

Reports are rendered later from the persisted run: `_assemble` (`api/routers/reports.py`) → `run_framework_view` (mappings by rulepack checksum, `api/comply/frameworks.py`) → `build_report` (`api/report/model.py`) → `render_html` / PDF (`api/report/render.py`, `api/report/pdf.py`), then a `REPORT_GENERATED` chain record.

3.2 The training loop, end to end

1. `GET /training/queue` → `training_queue` → `build_queue`: clusters residue and attaches up to three suggestions per cluster (`api/train/service.py`, `api/train/queue.py`).
2. `POST /training/confirm` → `confirm`: appends `ADMIN_CONFIRMED` or `ADMIN_CORRECTED` to the chain first, then stores a `TrainingExample` carrying that audit sequence.
3. `POST /training/compile` → `compile_confirmation` → `compile_pattern`, `draft_with_pattern`, `write_pack`: a DRAFT pack version with a checksum (`api/train/compile.py`, `api/train/activation.py`); `PACK_CREATED` recorded.
4. `POST /training/activate` → `activate_draft`: `validate` the draft, write the activation record, call `clear_pack_cache()`, record `PACK_ACTIVATED`. The next parse uses the new version with no restart.
5. `POST /training/withdraw` → `withdraw_pattern`: a new version without the pattern, activated; the version that contained it stays on disk for provenance. `POST /training/rollback` → `rollback_pack`: returns a platform to an earlier version exactly as it was, recording `PACK_ROLLED_BACK`.

3.3 The interface

`ui/` is a pure consumer: it computes no verdict, resolves no mapping and decides no scope. Pages (`ui/src/pages/`): Landing, Login, Devices, Compliance, Remediation, Reports, VendorPacks, Activity, Status. The device workspace (`ui/src/components/device/`) has tabs Overview, Findings, Needs review, Remediation and Report, with the benchmark selector (`BenchmarkScope.tsx`) above them; all state comes from one hook (`useDeviceWorkspace.ts`). Finding detail — evidence with surrounding lines, mapped controls with edition and provenance, declined mappings — is in `Findings.tsx` with `components/domain/EvidenceViewer.tsx`. API calls are in `ui/src/services/`; the training screen is `Review.tsx`.

4. Data model

4.1 Canonical fields

Thirteen named fields (`CANONICAL_FIELD_NAMES`, `api/models/csm.py`): `ssh_version`, `telnet_enabled`, `http_server_enabled`, `https_server_enabled`, `min_password_length`, `idle_timeout_seconds`, `logging_enabled`, `logging_hosts`, `ntp_servers`, `snmp_v3_only`, `banner_present`, `aaa_enabled`, `weak_ciphers`. The set is a reference, not an enforced schema, so a new field is a data change. Packs populate: Cisco IOS 8, Cisco NX-OS 7, JunOS 0, EOS 0 (`declared_fields`, `api/parse/fields.py`). Seven rules evaluate them (`rules/canonical/`):

RULE	FIELD	CONDITION	SEVERITY
`NRK-SSH-001`	ssh_version	equals 2	high
`NRK-TELNET-001`	telnet_enabled	is_false	critical
`NRK-HTTP-001`	http_server_enabled	is_false	high
`NRK-TIMEOUT-001`	idle_timeout_seconds	gt 0 and lte 600	medium
`NRK-LOGGING-001`	logging_hosts	non_empty	medium
`NRK-NTP-001`	ntp_servers	non_empty	low
`NRK-BANNER-001`	banner_present	is_true	low

4.2 The contracts

All in `api/models/`, frozen Pydantic models with `extra="forbid"`.

- **Structure:** `ConfigTree`, `ConfigNode`, `UnplacedLine` (config_tree); `FieldMatch`, `ParseResult` (parsing).
- **Facts:** `Field` — value, state, confidence, confidence method, evidence — and `Evidence` — file, line, raw line (field, evidence). Four states: PRESENT, ABSENT_DEFAULT, ABSENT_UNSUPPORTED, UNKNOWN. Five confidence methods: DETERMINISTIC, ADMIN_CONFIRMED, PLATFORM_DEFAULT, CALIBRATED_SIMILARITY, UNCALIBRATED_SIMILARITY.
- **Device:** `CanonicalSecurityModel`, `DeviceIdentity`, `CsmSource` (pack versions keyed `vendor/os_family`), `UnknownLine`, `Interface`, ACL types (csm, acl).
- **Verdicts:** `Finding` with `ObservedValue` and `FindingProvenance` — engine version, rulepack version and checksum, pack versions (finding). Four verdicts: PASS, FAIL, UNKNOWN, NOT_APPLICABLE. Eight UNKNOWN reasons, from `no_match` to `rule_type_mismatch`.
- **Rules:** `ComplianceRule`, `CheckSpec`, `Condition`, `AbsencePolicy`, `FrameworkRef`, `FrameworkGap`, `Rulepack` (rule).
- **Packs:** `VendorPack`, `PatternDef`, `DetectSignature`, `IdentityPattern`, `PlatformDefault`, `InterfaceRole` (pack).
- **Learning and audit:** `Suggestion`, `TrainingExample` (training); `AuditRecord`, `Actor`, `Subject` (audit); `RemediationSnippet` (snippet).

4.3 The two databases

STORE	TABLES (FROM `API/DB/MIGRATIONS/`)

`nirikshak.db` operational, 5 migrations	`config_file`, `config_line`, `line_cache`, `device`, `ingestion`, `app_user`, `audit_run`, `finding`, `finding_evidence`, `unknown` `schema_migrations`
`nirikshak-audit.db` audit, 1 migration	`audit_log`, `audit_chain_head`, `schema_migrations`; triggers `audit_log_no_update`, `audit_log_no_delete`

Why separate (ADR 0009, D4): the claim that the audit log contains no configuration content is provable by opening one file; the operational store can be protected and, later, encrypted without affecting the chain's verifiability. **The chain:** each entry hash is SHA-256 over canonical JSON of its sequence, timestamp, actor, action, subject, payload hash and the previous entry's hash (`compute_entry_hash`, `api/models/audit.py`), from a genesis of sixty-four zeros; `audit_chain_head` detects truncation of the tail. Writes are serialised by a lock (`api/audit/chain.py`). It is tamper-evident, not tamper-proof: an attacker with unrestricted write access could recompute the unkeyed chain (ADR 0008).

5. Testing and verification

2,566 tests are collected: 1,207 unit (`tests/unit/`), 515 integration (`tests/integration/`), 838 architecture (`tests/architecture/`) and 6 in `tests/test_health.py`. From a fresh clone with every optional group installed: 2,553 passed, 0 failed, 13 skipped. The documentation commits after `28ab52b` add one architecture test — `api/learn/` now bans `httpx` as every other package does (`test_learn_boundaries.py`) — so the suite there collects 2,567, 839 of them architecture, and gives 2,554 passed, 0 failed, 13 skipped. The interface has 54 tests in 9 files (`ui/src/test/`). The evaluation harness (`eval/`, output in `eval/reports/evaluation.txt`) scores the evaluation split against hand-written labels: Cisco field precision 100% over 20 assertions, recall 76.9%, FAIL precision 100% (7 of 7), FAIL recall 77.8% (7 of 9), and **zero wrong-confident answers** across all vendors.

5.1 Architecture tests enforce decisions rather than document them

- **Import boundaries:** 75 forbidden edges, statically scanned (`test_import_rules.py`).
- **No vendor logic in the engine:** `api/comply/` may not name a vendor or a canonical field (`test_comply_boundaries.py`).
- **No device access, no network clients** in ten packages — `analyse`, `comply`, `ingest`, `learn`, `normalise`, `parse`, `prioritise`, `remediate`, `report`, `train` (`test_no_device_libraries.py`, each `test_*_boundaries.py`).
- **No skip that silently stops testing:** every `skipif(C)` must be paired with `skipif(not C)` or registered with a reason (`test_skip_guards.py`); a shadowed test fails the suite (`test_no_shadowed_tests.py`).
- **Documents answerable to code:** the defect register, decision index, framework identifiers, mapped counts and eleven registered prose claims are reconciled against the code (`test_architecture_document.py`, `test_framework_claims.py`, `test_prose_claims.py`).
- **Content policy:** no CIS benchmark material anywhere in the tree (`test_cis_material_is_not_held.py`); no transcribed framework prose in rules (`test_rule_content_policy.py`).

5.2 When the tool caught its own authors

1. **A fabricated model shipped with a citation.** The Cisco pack read the hardware model from `! model ISR4331`, a comment a corpus author typed (`corpus/cisco/dev/rtr-core-01.cfg`, line 6), and reported it as observed identity with a citation. Cisco does not write the model into a running configuration. The pattern was removed. Recorded in ADR 0037.
2. **An ACL remark the analyser contradicted.** `edge-rtr-01.cfg` carried "*this deny is unreachable: the catch-all permit below already matches it*"; the analyser declined to flag the entry. Access lists evaluate top-down, so a rule below cannot shadow one above. The analyser was right; the fixture was fixed. ADR 0028.
3. **A term named for what it did not do.** In `core-rtr-01.conf`, `term allow-established` matched *all* TCP, and the analyser reported the management permits beneath it as redundant. It was right about the configuration. The name was changed. ADR 0050.
4. **Four framework mappings declined.** A prepared reading mapped rules to DISA STIG controls the rules are looser than, which would have printed a STIG identifier beside a PASS the STIG calls a finding: idle timeout (rule allows up to 600 s, CISC-ND-000720 requires 300 s or less), logging (one host against two), NTP (one server against redundant sources) and the banner (presence against mandated text). All four were declined, each with its reason recorded on the rule (`not_mapped` in `rules/canonical/*.yaml`). ADR 0052.
5. **Verification that depended on where it ran.** A fresh-clone run reported 0 failures when it had 8: `pytest -rs` prints skipped tests but not failures, and the check reading the log looked only for failure summaries. The run exposed two real defects invisible from the machine that wrote the code — a corpus-manifest hash recorded from a CRLF working copy (so every fresh clone failed the integrity test), and model-refusal tests sensitive to a model cached machine-wide. The same discipline had found an interface defect:

running the interface in a real browser showed the benchmark-scope sentence printing a pattern through Python's `repr`, and writing a test for the audit refusal showed that every audit failure in the interface had been silent. Recorded in ADR 0060 and ADR 0058. The lesson is that verification is only as good as the environment it runs in — and this project's argument is that its checks are real.

6. Feasibility, viability and impact

6.1 Feasibility — the design at scale

The parse is per-file and stateless. `parse_configuration` and `build_csm` take one file's text and one pack and share no state between files, so files can be processed independently and in parallel. **The verdict path holds no model.** `evaluate_device` is a deterministic function of the canonical model and the rulepack; its cost does not depend on any model being present. **Coverage grows without redeployment:** packs, rules and framework indexes are data, and a pack is activated at runtime (`clear_pack_cache`). **It runs fully offline:** no device access, no network client in the pipeline, the model loader fails closed under airgap, and fonts are vendored — which is what air-gapped and defence networks require. **Runs are attestable:** every audit, confirmation, pack change and report is a hash-chained record.

Where the bottleneck would sit, and what moves first. The audit chain is serial by construction — each record depends on its predecessor, and appends take a lock (`api/audit/chain.py`) — and SQLite admits one writer at a time. At fleet scale the first thing to move is therefore persistence: the operational store to a server database, with chain appends batched per run, keeping the chain itself serial because that is what makes it a chain. Second, retrieval: suggestion ranking compares a query with every labelled example (`rank_candidates`); as administrator confirmations accumulate, that moves to an approximate nearest-neighbour index such as FAISS. Parsing and rule evaluation, being per-file and stateless, scale out rather than up.

6.2 Viability

Adding a platform is a YAML file, not a vendor release cycle: detection signatures, identity patterns and field patterns in one pack. That first pack is written by hand: a file whose platform detection cannot place answers 409 at audit and never reaches the training queue (`run_audit_endpoint`), and `compile_confirmation` extends an existing pack but cannot create one. From then on the platform grows through an administrator working the *Needs review* queue, not a developer: each confirmation compiles into a readable, anchored pattern that is reviewed before activation. The same holds for frameworks (an index file) and rules (a YAML check).

6.3 Impact

Misconfiguration is among the leading causes of network security incidents, and the problem statement names network devices as the most common point of misconfiguration. An auditor whose every claim cites a configuration line — and which says UNKNOWN, with a reason, rather than guessing — produces output that survives being questioned. Offline operation means it can be used where configuration data cannot leave the network.

7. Known limitations

7.1 Open defects (`docs/architecture.md` §9)

- **DEF-3** — `device_id` is the configuration file's content hash, so it identifies a configuration, not a physical device over time; a re-uploaded, edited file counts as a second device.
- **DEF-16** — the confirmation loop has no notion of corpus splits, so an administrator can compile a pattern from a file reserved for evaluation. A guard now detects it; nothing prevents it.
- **DEF-18** — deleting a trained pack orphans stored findings that cite it; the pack files are archived (`packs/archive/`) but no loader resolves a version through the archive.
- **DEF-19** — a second account uploading a file another account uploaded first sees the device and is refused its audit (404), because a file's owner is its first uploader.
- **DEF-20** — **AI suggestions are not chain records.** DEVELOPER_GUIDE.md §9 requires a hash-chained record of AI suggestions. `AuditAction.AI_SUGGESTED` is defined, and the audit database permits a model actor that action and no other, but nothing appends one. The chain records each administrator decision instead (`ADMIN_CONFIRMED` / `ADMIN_CORRECTED`, `confirm` in `api/train/service.py`) with the number of suggestions shown and whether the chosen field was among them; the suggestions themselves are stored in `training_example.suggestions_json`, outside the chain. That count is recomputed from the training queue when `/training/confirm` is called (`api/routers/training.py`), not recorded from what the screen displayed, so it attests what the server held for that cluster at the moment of the decision.
- **DEF-21** — **encryption at rest is not implemented.** DEVELOPER_GUIDE.md Rule 6 requires it. The operational database holds every uploaded line and `uploads/` holds every file verbatim, secrets included — scrubbing happens before inference, not before storage, so evidence can quote the operator's own line. Both are plain files. `api/config.py` and `api/ingest/blobs.py` describe the blob store as what decision R11 "would" encrypt; R11 was never taken. Until it is, the host must protect those files (disk encryption, file permissions). For a security tool this is the most serious open item, and it is stated here without qualification.

7.2 Sourcing gaps (`docs/SOURCING_BACKLOG.md`)

- Vendor capability and default documentation — no platform default ships, so absence-aware evaluation has not run on real data; no pack declares management interfaces, so exposure ranking abstains on every finding.
- An XML sample independent of the PAN-OS holdout — blocks the XML syntax mode and the held-out generalisation experiment.
- ISO/IEC 27001 — a purchased standard; not sourced.
- Broader vendor diversity — JunOS and EOS read no canonical field; STIG and CIS editions cover Cisco IOS XE 17.x only.
- Vendor remediation documentation for NX-OS — no snippet exists.
- Line-level ground truth — top-3 accuracy and calibration are not measured.
- A corpus by more than one author — every corpus file is hand-written by the team; nothing here is a field-accuracy claim.

7.3 Partial requirements (`docs/PS26155_COMPLIANCE.md`)

- R2 — the training interface extends a platform that already has a pack; it cannot teach an unseen vendor, which the official statement asks for. Suggestions need the optional `[ai]` extra and are never calibrated.
- A1 — the statement asks an AI-based engine to normalise and to analyse deviation; here both are deterministic, by design (Rule 1).
- Hardening examples — strong cryptographic suites and logging of administrative access are not checked; `weak_ciphers` is in the model and read by no pack.

- R3 — three of four frameworks; ISO absent.
- R4 — no serial numbers (not in a configuration export; ADR 0044); hardware model on EOS only; no NX-OS remediation.
- R5 and "any vendor" — XML and JSON configuration formats are not parsed.

7.4 Found while preparing this report

- DEF-20 and DEF-21 above were found by comparing DEVELOPER_GUIDE.md with the code, and entered in the register; DEVELOPER_GUIDE.md now states each rule and then the fact.
- **FAISS is installed and not used** (section 1). DEVELOPER_GUIDE.md §11 no longer lists it.

8. Index: concept to file

ACL interval analysis	`api/analyse/intervals.py`, `api/analyse/acl_analysis.py`
Absence resolution	`api/normalise/absence.py`
Audit chain (append, verify)	`api/audit/chain.py`, `api/audit/verify.py`, `scripts/verify_audit_chain.py`
Audit request route	`api/routers/audits.py` — `run_audit_endpoint`
Benchmark selector (UI)	`ui/src/components/device/BenchmarkScope.tsx`
Block parser	`api/parse/block_parser.py`
Calibration refusal	`api/learn/calibration.py`
Canonical model	`api/models/csm.py`, `api/normalise/service.py`
Clustering, signatures	`api/learn/cluster.py`, `api/learn/signature.py`
Confirmation loop	`api/train/service.py`, `api/routers/training.py`
Contracts	`api/models/`
Databases and migrations	`api/db/`, `api/db/migrations/`
Defect register	`docs/architecture.md` §9
Embedding adapter	`api/learn/embedding.py`
Evaluation harness	`eval/`, `eval/reports/evaluation.txt`
Evidence	`api/models/evidence.py`, `api/models/field.py`
Forbidden import edges	`tests/architecture/test_import_rules.py`
Framework indexes, scope	`rules/frameworks/`, `api/comply/frameworks.py`
Holdout (sealed)	`corpus/holdout/panos/`, `eval/corpus.py`
Identity extraction	`api/ingest/device_identity.py`
Ingestion, upload	`api/ingest/service.py`, `api/routers/ingest.py`
Merge policy	`api/models/csm.py` (`FIELD_MERGE_POLICY`), `api/parse/fields.py`
Pack activation, lifecycle	`api/train/activation.py`, `api/ingest/packs.py`
Pack checksums	`api/ingest/pack_checksum.py`
Pattern compiler	`api/train/compile.py`
PDF and HTML reports	`api/report/`, `api/routers/reports.py`
Remediation	`snippets/`, `api/remediate/resolver.py`
Review screen (UI)	`ui/src/components/device/Review.tsx`
Rule engine	`api/comply/engine.py`, `api/comply/conditions.py`
Rulepack and checksum	`rules/canonical/`, `rules/rulepack.yaml`, `api/comply/rulepacks.py`
Secret scrubbing	`api/security/scrub.py`
Similarity ranking	`api/learn/suggest.py`, `api/learn/index.py`
Vendor detection	`api/ingest/vendor_detect.py`
Vendor packs	`packs/builtin/`